

Software Requirements Specification

for

Inter NIT Sports Management System

Version 3.0

Prepared by

Team Number: 1

Srinath M P	**M251063CS**
Harish Rana	**M250785CS**
Virendrasing Patil	**M250560CS**
Achyut Prabhakar	**M250617CS**

Course: CS6103E - Software Systems lab

Date: 22-09-2025

1 Introduction **3**
1.1 Document Purpose 3
1.2 Product Scope 3
1.3 Intended Audience and Document Overview 3
1.4 Definitions, Acronyms and Abbreviations 3
1.5 Document Conventions 3
1.6 References and Acknowledgments 4
2 Overall Description **4**
2.1 Product Overview 4
2.2 Product Functionality 4
2.3 Design and Implementation Constraints 4
2.4 Assumptions and Dependencies 5
3 Specific Requirements **5**
3.1 External Interface Requirements 5
3.1.1 User Interfaces 5
3.1.2 Hardware Interfaces 5
3.1.3 Software Interfaces 5
3.2 Functional Requirements 6
3.3 Use Case Model 6
3.3.1 Use Case #1 6
3.3.2 Use Case #2 7
4 Other Non-functional Requirements **7**
4.1 Performance Requirements 7
4.2 Safety and Security Requirements 7
4.3 Software Quality Attributes 8
5 Other Requirements **8**
Appendix A - Activity Log **8**

1 Introduction

1.1 Document Purpose

The purpose of this document is to specify the functional and non-functional requirements of the Inter-NIT Sports Management System (INSMS). This system is designed to facilitate the management of the annual Inter-NIT tournament by providing a centralized web-based platform that allows Common Admins, NIT Admins, and Coaches to efficiently register teams, manage events, publish schedules, and record results.

This document is intended for developers (MERN stack team), testers, project managers, and stakeholders of the INSMS project.

1.2 Product Scope

The INSMS provides a digital solution for managing the Inter-NIT Sports Tournament. The system's main features include:

- NIT registration and approval by a Common Admin.
- Team management through Coaches and NIT Admins.
- Event scheduling and venue allocation by Hosting NIT Admins.
- Accommodation and mess details publishing.
- Result recording and publishing.
- Notifications and leaderboards for all stakeholders.

The product will be a web-based application built with the MERN stack, ensuring scalability, role-based security, and ease of use.

1.3 Intended Audience and Document Overview

The intended audience for this Software Requirements Specification (SRS) document includes developers, project managers, testers, and stakeholders involved in the creation and deployment of the "Sports Management System". Additionally, it is relevant for college administrators who oversee the platform's use on campus, as well as future maintenance teams. This document provides a comprehensive overview of the functional and non-functional requirements, system architecture, user roles, and interface design. Its purpose is to ensure that all parties have a clear understanding of the system's goals, features, and implementation requirements to achieve a successful and user-friendly platform for college students

1.4 Definitions, Acronyms and Abbreviations

- **INSMS** – Inter-NIT Sports Management System
- **NIT** – National Institute of Technology
- **Admin** – Authorized administrator of the system
- **CRUD** – Create, Read, Update, Delete
- **JWT** – JSON Web Token (authentication mechanism)
- **MERN** – MongoDB, Express.js, React.js, Node.js

1.5 Document Conventions

This Software Requirements Specification (SRS) document adheres to standard IEEE formatting requirements to maintain clarity and consistency. The following conventions are used throughout this document:

- **Formatting Conventions:** All text is written in Arial font, with a size of either 11 or 12, depending on readability needs. The document is single-spaced with 1" margins on all sides. Section and subsection titles are formatted according to the template structure and use bold headings for clear navigation. Comments or additional notes are italicized to differentiate them from the main text.
- **Naming Conventions:** Standard naming conventions are used to maintain clarity. System entities such as "User," and "Notification" are capitalized when referenced as distinct components. Descriptive headings and subheadings are utilized to structure the document effectively.

1.6 References and Acknowledgments

MongoDB Inc., MongoDB Documentation.
Express.js Documentation
React Documentation
Node.js Documentation

2 Overall Description

2.1 Product Overview

The Inter-NIT Sports Management System (INSMS) is a new, self-contained web application built on the MERN stack (MongoDB, Express.js, React.js, Node.js). It is designed to provide a centralized platform for organizing and managing the annual Inter-NIT sports tournament. The system replaces traditional manual processes such as paper registrations, email-based communication, and offline result recording, offering a digital and efficient alternative.

The product operates independently but may interact with external services such as email/SMS APIs for notifications, and cloud-based deployment environments like MongoDB Atlas and hosting platforms (Heroku, AWS, or Netlify). It does not replace an existing enterprise solution but introduces a modern, scalable approach for inter-collegiate tournament management.

2.2 Product Functionality

At a high level, the major functions of INSMS include:

- NIT registration and approval by Common Admin.
- Team registration by Coaches.
- Event venue allocation by NIT Admins.
- Publication of accommodation and mess information.
- Recording and publishing match results.
- Generating leaderboards and reports.
- Sending notifications (email/SMS) to stakeholders.

2.3 Design and Implementation Constraints

The system must operate under the following constraints:

- **Technology**: Implementation restricted to MERN stack (MongoDB, Express.js, React.js, Node.js).
- **Authentication**: JWT-based role-specific access control.
- **Deployment**: Must run on cloud hosting platforms (AWS, Heroku, or Netlify).
- **Fixtures**: Match fixtures will be created manually, not auto generated.
- **Browser Support**: Only modern browsers (Chrome, Firefox, Edge) will be supported.
- **Security**: Passwords must be encrypted; all communication via HTTPS.

2.4 Assumptions and Dependencies

The following assumptions and dependencies are considered in the design:

- Each NIT will have at least one registered Admin and Coach responsible for data entry.
- Hosting NITs will provide accurate schedules, venue, and accommodation details.
- Users will have stable internet connectivity during tournament usage.
- External APIs such as SMTP/email and SMS gateways will be available and reliable.
- The system depends on MongoDB Atlas for database hosting.
- Future features (like AI-based fixture generation or live score integration) may require integration with third-party services.

3 Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

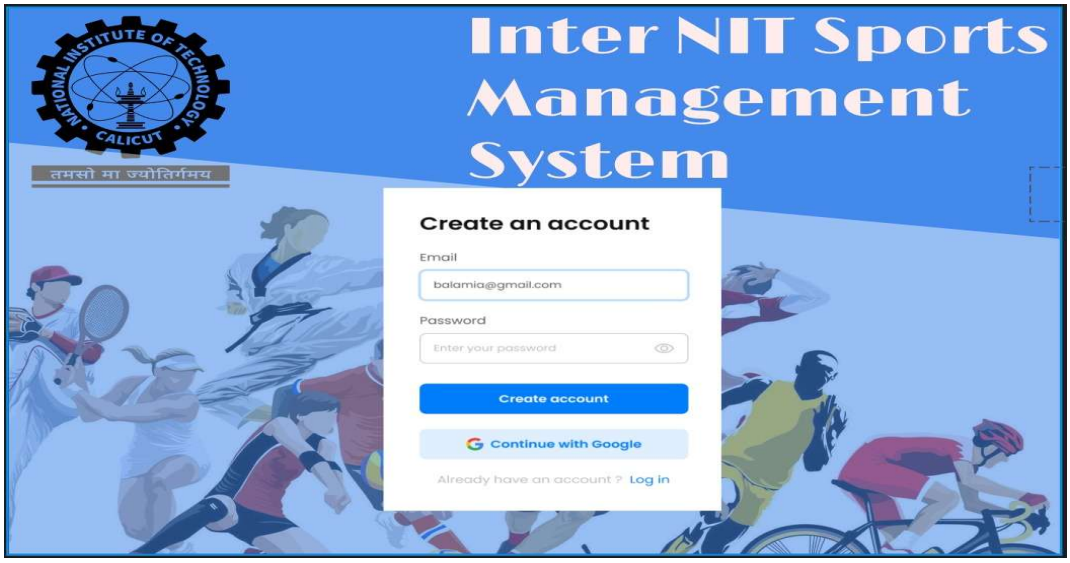
The INSMS web application provides role-based dashboards and interactive interfaces for all types of users: Common Admin, NIT Admin, and Coaches. Each dashboard is designed to be intuitive and responsive across devices. Users interact with the system through forms, tables, and menus for operations such as team registration, event scheduling, result entry, and viewing notifications.

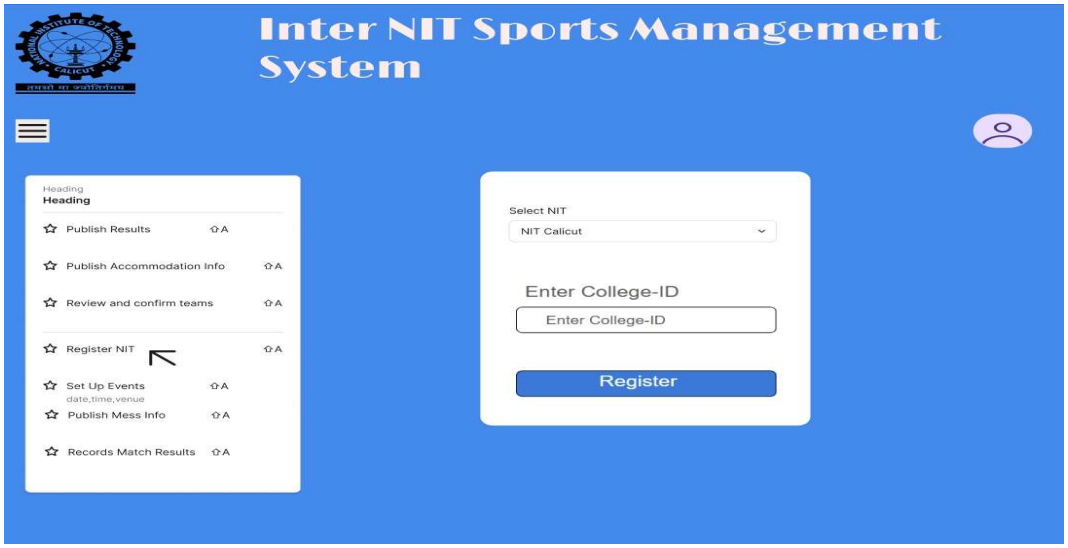
GUI Overview:

- **Common Admin Dashboard:** Approve NIT registrations, validate event entries, manage fixture schedules.
- **NIT Admin Dashboard:** Manage teams, create events, publish venue and accommodation information.
- **Coach Dashboard:** Register players, view team roster, check match schedules, view results.
- **Public View:** Display published results, schedules, and announcements.









3.1.2 Hardware Interfaces

The system is primarily web-based and accessed via standard desktops, laptops, and mobile devices. No specialized hardware is required.

- Optional Future Interface: If the mess/accommodation module is extended to use fingerprint scanners or IoT devices, the software should support basic integration APIs for data collection.

3.1.3 Software Interfaces

INSMS interacts with the following external software components:

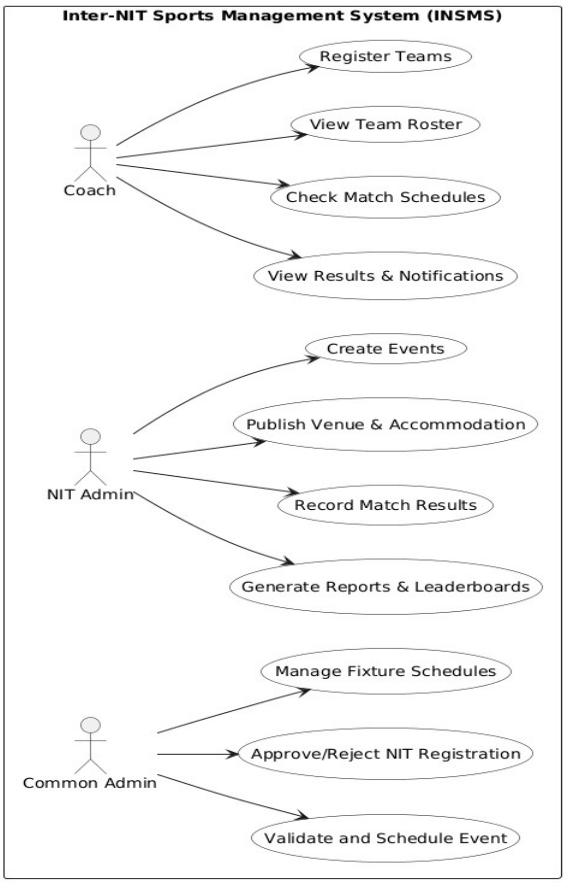
- **MongoDB Atlas**: Cloud database for storing user data, events, teams, and results.
- **Email/SMS API**: Used for sending notifications to users.
- **Web Browsers**: Google Chrome, Firefox, and Edge for accessing the web application

3.2 Functional Requirements

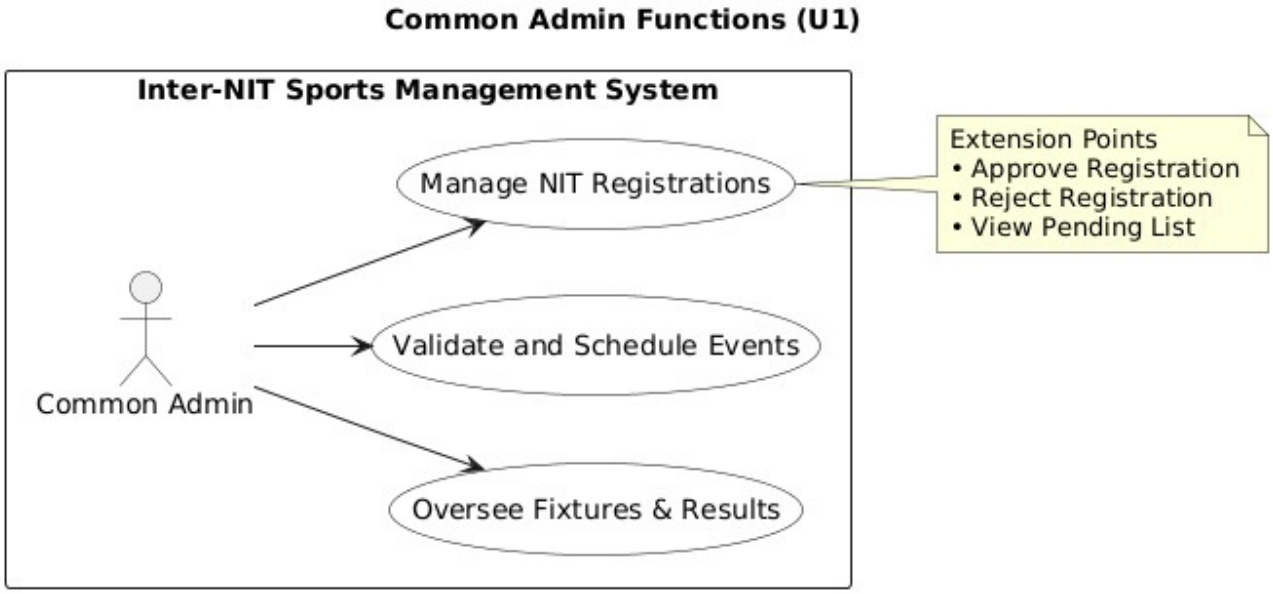
The system shall support the following functional requirements:
F1: The system shall allow Common Admins to approve or reject NIT registrations.
F2: The system shall allow Coaches to register teams and players.
F3: The system shall allow NIT Admins to create Events with venue and time.
F4: The system shall publish accommodation and mess details for hosting NITs.
F5: The system shall allow NIT Admins to record match results including winner, score, and remarks.
F6: The system shall generate leaderboards and reports for teams and players.
F7: The system shall send notifications to users via email or SMS.
F8: The system shall provide role-based access control for all functionalities.
F9: The system shall store all team and match data securely in MongoDB.
F10: The system shall display schedules, results, and announcements on user dashboards.
F11: The System shall allow Common Admin to schedule events requested by host-NIT Admin.
F12: The system shall maintain an audit trail of all administrative changes.
F13: The system shall allow Common Admins to manually set fixture pairings.
F14: The system shall provide an export option for reports in PDF or Excel or JSON format.
F15: The system shall validate all input data to prevent errors or inconsistencies.

3.3 Use Case Model

The INSMS use case model encapsulates all actors and system interactions. The main actors are Common Admin, NIT Admin, and Coach.



3.3.1 U1: Common Admin – Approve/Reject NIT Registration



Purpose - The objective of this use case is to enable the Common Admin to approve or reject NIT registrations. This ensures that only valid and authorized NITs participate in the Inter-NIT tournament.

Requirements Traceability –

F1: The system shall allow Common Admins to view pending NIT registrations.

F2: The system shall allow Common Admins to approve or reject registrations.

F3: The system shall notify NIT Admins about the decision.

Priority - High – critical for validating institutions before they can participate.

Preconditions - Common Admin must be authenticated and logged in. Pending NIT registration requests must exist in the system.

Post conditions - Approved NITs gain access to the system. Rejected NITs are notified and cannot proceed further.

Actors – Common Admin (primary), System (secondary).

Flow of Events

1. **Basic Flow:**
    - Common Admin logs into the system.
    - The system displays pending NIT registration requests.
    - Common Admin reviews registration details.

- Common Admin selects “Approve” or “Reject.”
- Common admin approves or rejects events.
- Common Admin schedules events.
- The system updates the NIT status and sends a notification to the respective NIT Admin.

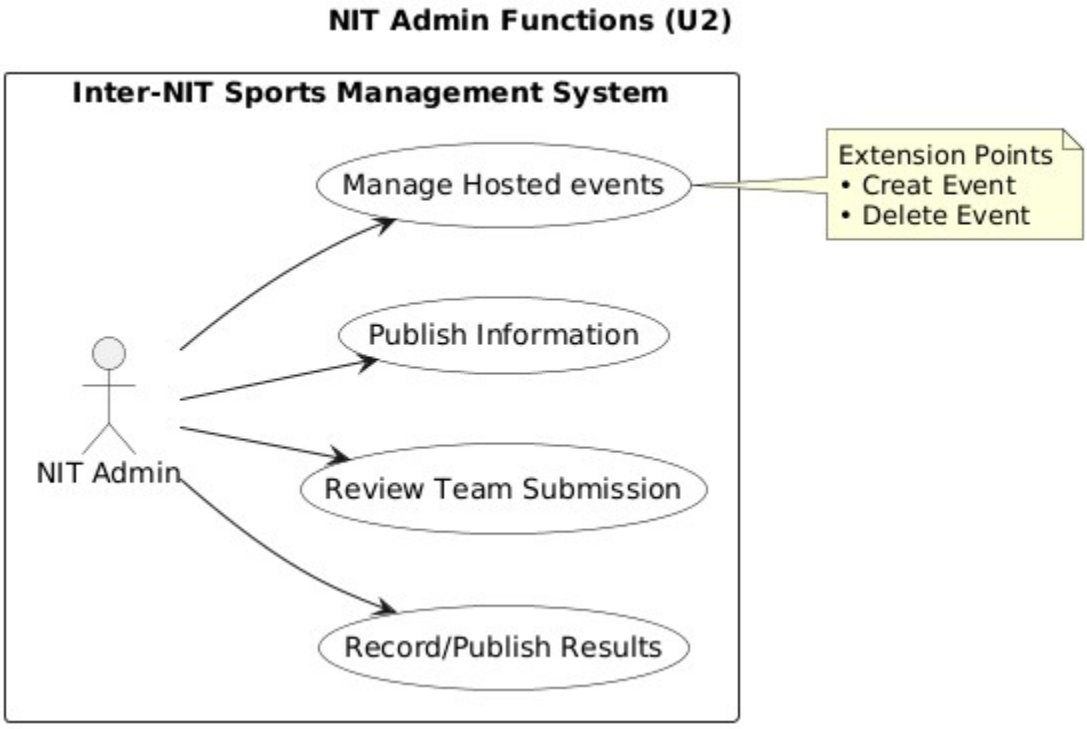
2. **Alternative Flow:**
   - AF1: If no registrations are pending, the system displays “No pending registrations.
3. **Exceptions**:
   - E1: If the system fails to update approval status due to a server error, an error message is displayed.

Includes: None

Excludes: None

Notes/Issues - Future updates may include bulk approval or AI-based validation of NIT documents.

3.3.2 U2: NIT Admin – Publish Information



Purpose - The objective of this use case is to allow NIT Admins (especially from hosting institutes) to allocate venues, and publish accommodation and mess details.

Requirements Traceability –

F3: The system shall allow NIT Admins to create events.

F4: The system shall publish accommodation and mess details.

F5: The system shall allow NIT Admins to record match results.

Priority - High – ensures tournament logistics and smooth operations.

Preconditions - NIT Admin must be authenticated. Events must be defined for the tournament.

Post conditions - Events venues are published. Accommodation and mess details are visible to participants. Results recorded by NIT Admins are stored in the system.

Actors – Common Admin (primary), System (secondary).

Flow of Events

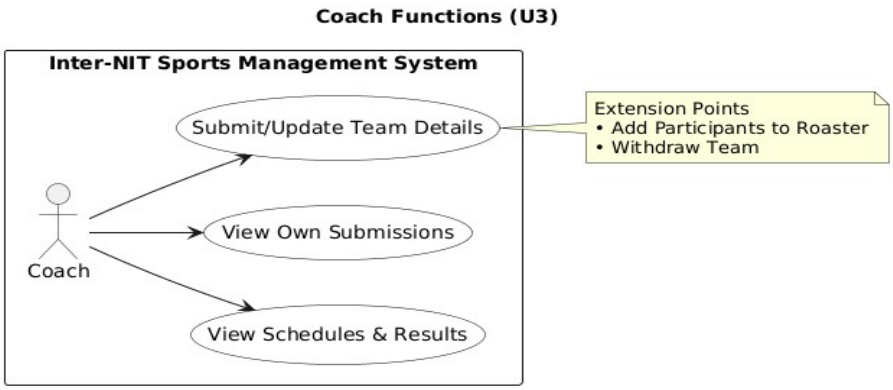
1. **Basic Flow:**
    - NIT Admin logs in.
    - NIT Admin navigates to “Event Management.”
    - The system displays upcoming events and scheduling options.
    - NIT Admin sets venue for matches.
    - NIT Admin publishes accommodation and mess information.
    - The system stores details and makes them available to all stakeholders..
2. **Alternative Flow:**
    - AF1: If no venues are available, the system displays “Venue not available.”
3. **Exceptions**:
    - E1: Database connectivity error prevents saving schedule details.

Includes: Record Match Results (U3).

Excludes: May extend “View Event Details” by participants.

Notes/Issues - Future integration may include auto-scheduling of matches or venue availability.

3.3.3 U3: Coach – Register Teams & Players



Purpose - The objective of this use case is to allow Coaches to register teams, manage team rosters, and view schedules/results.

Requirements Traceability –

F2: The system shall allow Coaches to register teams.

F10: The system shall display schedules and results.

Priority - High – Coaches are essential for managing teams.

Preconditions - Coach must be authenticated. Team details must be available for registration.

Post conditions - Teams are stored in the system. Coaches can access updated schedules and results.

Actors – Coach (primary), System (secondary).

Flow of Events:

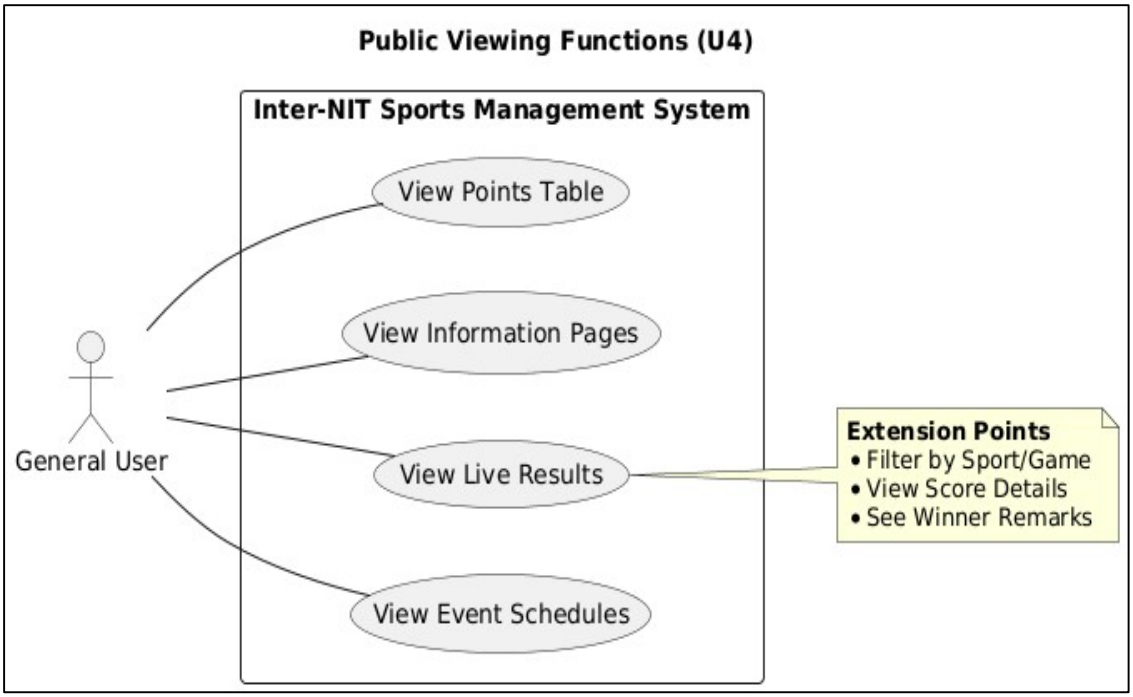
1. **Basic Flow:**
    - Coach logs into the system.
    - Coach navigates to “Team Management.”
    - The system displays team registration form.
    - The system validates and saves the team roster.
    - Coach views schedules and results from the dashboard.
2. **Alternative Flow:**
    - AF1: If team roster exceeds limit, the system shows an error.
3. **Exceptions:**
    - E1: Server error prevents saving team details.

Includes: View Match Schedules and Results.

Excludes: May extend “Update Player Information.”

Notes/Issues - Future updates may allow bulk upload of team rosters via Excel/CSV.

3.3.4 U4: Student/User – View Results, Announcements, and Leaderboard



Purpose - The objective of this use case is to allow general users (students, spectators, and participants) to view published results, announcements, and leaderboards.

Requirements Traceability –

F10: The system shall display schedules, results, and announcements.

F6: The system shall generate leaderboards and reports.

F7: The system shall send notifications to users.

Priority - Medium – provides visibility to all stakeholders but is dependent on data entered by admins and coaches.

Preconditions - The system must have published results and schedules. User must have internet access.

Post conditions - Users can view updated results and announcements. Leaderboards are dynamically updated.

Actors – Student/User (primary), System (secondary).

Flow of Events

1. **Basic Flow:**
    - User navigates to the public dashboard (no login required).
    - The system displays the latest schedules, results, and announcements.
    - User clicks on “Leaderboard” to view rankings.

- The system retrieves and displays leaderboard details.

2. **Alternative Flow:**
   - AF1**:** If no events are scheduled, the system displays “No events available.”
   - AF2: If leaderboards are empty, the system shows a placeholder message.
3. **Exceptions:**
   - E1: Database query fails, preventing results display.

Includes: None

Excludes: None

Notes/Issues - In future, live score updates and mobile notifications may be added.

4 Other Non-functional Requirements

4.1 Performance Requirements

The Inter-NIT Sports Management System (INSMS) is expected to perform efficiently under various conditions to ensure a smooth user experience for all stakeholders. The performance requirements are as follows:

- The system shall support at least 500 concurrent users without performance degradation.
- All dashboard updates, including schedules, results, and announcements, shall reflect within 2 seconds of entry.
- Data retrieval operations (e.g., fetching team roster, match results) shall complete within 1 second under normal load.
- Batch operations, such as exporting reports or generating leaderboards, shall complete within 5 seconds for up to 1000 entries.
- Notifications (email/SMS) triggered by the system shall be dispatched within 30 seconds of the triggering event.

4.2 Safety and Security Requirements

INSMS must ensure the safety, privacy, and security of all data and system operations:

- The system shall implement role-based access control to prevent unauthorized access to sensitive data.
- All user passwords shall be stored in encrypted form using modern hashing algorithms (e.g., crypt).
- The system shall transmit all data over HTTPS to prevent interception or data leakage.
- Backup of the database shall occur every 24 hours to prevent data loss.
- The system shall validate all inputs to prevent malicious injections, incorrect data entry, and system crashes.
- User authentication shall require a valid username and password; optional multifactor authentication may be added for high-security operations.
- Access to administrative features shall be logged to maintain an audit trail for accountability.

4.3 Software Quality Attributes

Reliability:
The system must guarantee 99.9% uptime, ensuring high availability for users across different time zones. Redundant systems, such as backup servers, should be in place to handle failover in the event of system downtime or technical failures.
Usability:
The user interface shall be intuitive and user-friendly, ensuring that both students and instructors can easily navigate the platform. Dashboards for all user roles shall be intuitive and responsive, providing easy navigation with minimal clicks.
Adaptability:
The system must be accessible on multiple devices, including desktops, tablets, and mobile phones. A responsive design will ensure that the layout adjusts to different screen sizes and orientations.
Interoperability:

The system must be compatible with various third-party tools and services, such as email providers (for notifications), content delivery networks (for video streaming), and cloud storage. APIs should be designed for future integrations with external learning platforms or student management systems.

Maintainability:

The codebase should be well-structured and modular to facilitate ease of updates and bug fixes. Proper documentation for both developers and system administrators shall be maintained to ensure that the system can be easily modified and scaled as the user base grows.

Portability:

The application must be capable of running in different environments (e.g., development, testing, production) without requiring significant changes to the codebase. It should be deployable on cloud platforms (e.g., AWS, Azure) or on-premise servers.

5 Other Requirements

- **Legal Compliance:**
  The system must comply with international data protection laws, such as GDPR (General Data Protection Regulation) for users in the European Union. User consent must be obtained before storing or processing personal information, and users must have the right to delete their accounts and associated data.

Appendix A - Activity Log

Date	Duration	Meeting Details / Owner	Team Member Contributions
2025-09-21	4 hours	Initial meeting to define project scope and outline SRS structure. Led by Team Lead.	Defined project scope, structured SRS sections, assigned roles.